DOI:10.1145/3024918

Article development led by acmqueue
queue.acm.org

Hardware and software perspectives.

BY MOHAMED ZAHRAN

Heterogeneous Computing: Here to Stay

MENTIONS OF THE phrase *heterogeneous computing* have been on the rise in the past few years and will continue to be heard for years to come, because heterogeneous computing is here to stay. What is heterogeneous computing, and why is it becoming the norm? How do we deal with it, from both the software side and the hardware side? This article provides answers to some of these questions and presents different points of view on others.

Let's start with the easy questions. What is heterogeneous computing? In a nutshell, it is a scheme in which the different computing nodes have different capabilities and/or different ways of executing instructions. A heterogeneous system is therefore a parallel system (single-core systems are almost ancient history). When multicore systems appeared, they were homogeneous—that is, all cores were similar. Moving from sequential programming to parallel programming, which used to be an area only for niche programmers, was a big jump. In heterogeneous computing, the cores are different.

Cores can have the same architectural capabilities—for example, the same hyperthreading capacity (or lack thereof), same superscalar width, vector arithmetic, and so on. Even cores that are similar in those capabilities, however, have some kind of heterogeneity. This is because each core now has its own DVFS (dynamic voltage and frequency scaling). A core that is doing more work will be warmer and hence will reduce its frequency and become, well, slower. Therefore, even cores with the same specifications can be heterogeneous. This is the first type of heterogeneity.

The second type involves cores with different architectural capabilities. One example is a processor with several simple cores (for example, single-issue, no out-of-order execution, no speculative execution), together with a few *fat* cores for example, with hyperthreading technology, wide superscalar cores with out-of-order and speculative execution).

These first two types of heterogeneity involve cores with the same execution model of sequential programming—that is, each core *appears* to execute instructions in sequence even if under the hood there is some kind of parallelism among instructions. With this multicore machine, you may write parallel code, but each thread (or process) is executed by the core in a seemingly sequential manner. What if computing nodes are included that don't work like that? This is the third type of heterogeneity.

In this type of heterogeneity the computing nodes have different execution models. Several different types of nodes exist here. The most famous is the GPU (graphics processing unit), now used in many different applications beside graphics. For example, GPUs are used a lot in deep learning, especially the training part. They are also used in many scientific applications and are delivering performance that is orders of magnitude better than traditional cores. The reason for this performance boost is that a GPU uses the single-instruction (or thread), mul-

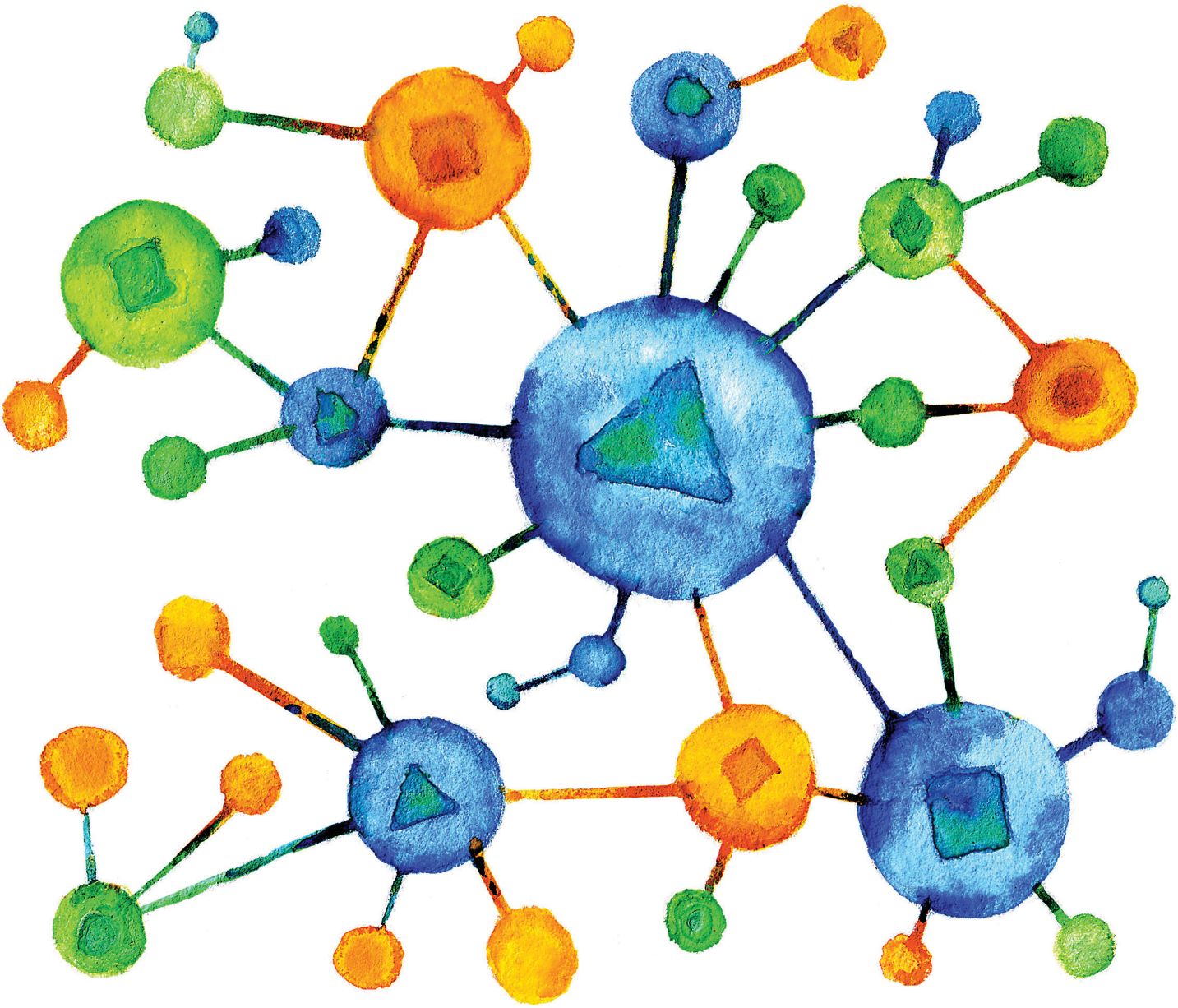


IMAGE BY ALICIA KUBISTA/ANDRIJ BORYS ASSOCIATES

tiple-data execution model. Let's assume you have a large matrix and need to multiply each element of this matrix by a constant. With a traditional core, this is done one element at a time or, at most, a few elements at a time. With a GPU, you can multiply all the elements at once, or in a very few iterations if the matrix is very large. The GPU excels in similar independent operations on large amounts of data.

Another computing paradigm that deviates from the traditional sequential scheme is the FPGA (field-programmable gate array). We all know that software and hardware are logically equivalent, meaning what you can do with software you can also do with hardware. Hardware solutions are much faster but inflexible. The FPGA tries to close this gap. It is a circuit that can be configured by the programmer to implement a certain function. Suppose you need to calculate a polynomial function on a group of elements. A single polynomial function is compiled to tens of assembly instructions. A FPGA is a good choice if the number of elements needed to calculate the function is not large enough to require a GPU, and not small enough to be done in a traditional core efficiently. FPGAs have been used in many high-performance clusters. With Intel's acquisition last year of Altera, one of the big players in the FPGA market, tighter integration of FPGAs and traditional cores is expected. Also, Microsoft has started using FPGAs in its datacenter (Project Catapult).

A new member recently added to the computing-node options is the AP (Automata processor) from Micron.[3] AP is very well suited for graph analysis, pattern matching, data analytics, and statistics. Think of it as a hardware regular expressions accelerator that works in parallel. If you can formulate the problem at hand as a regular expression, then you can expect to get much higher performance than a GPU could provide. AP is built using FPGAs but designed to be more efficient in regular expressions processing.

Aside from the aforementioned computing nodes, there are many other processing nodes such as the DSP (digital signal processor) and ASIC (application-specific integrated circuit). Those target small niches of applica-

tions, however, and are not as versatile as the ones mentioned earlier. Brain-inspired neuromorphic chips, such as IBM's TrueNorth chip, are starting an era of cognitive computing.[2] Cognitive computing, championed by IBM's Watson and TrueNorth, is now used, after the impressive performance of the AI computer system Watson on "Jeopardy," in medical applications, and other areas are being explored. It is a bit early, however, to compare it with the other more general-purpose cores.

The rest of this article considers only traditional cores (with different capabilities), GPU, FPGA, and AP. The accompanying figure shows the big picture of a heterogeneous computing system, even though, because of the cost of programmability, finding a system with the level of heterogeneity shown in the figure is unlikely. A real system will have only a subset of these types.

What is the advantage of having this variety of computing nodes? The answer lies in performance and energy efficiency. Suppose you have a program with many small threads. The best choice in this case is a group of small cores. If you have very few complicated threads (for example, complicated control-flow graphs with pointer-chasing), then sophisticated cores (for example, fat superscalar cores) are the way to go. If you assign the complicated threads to simple cores, the result is poor performance. If you assign the simple threads to the sophisticated cores, you consume more power than needed. GPUs have very good performance-power efficiency for applications with data parallelism. What is needed is a general-purpose machine that can execute different flavors of programs with high performance-power efficiency. The only way to do this is to have a heterogeneous machine.[3] Most machines now, from laptops to tablets to smart phones, have heterogeneous architectures (several cores and a GPU), and more heterogeneity is expected in the (very) near future. How should we deal with this paradigm shift from homogeneity to heterogeneity?

Hardware Challenges

Several challenges exist at the hardware level. The first is memory hierarchy. The memory system is one of the main performance bottlenecks in any computer system. While processors had been following Moore's Law until a few years ago, making good leaps in performance, memory systems have not. Thus, there is a large performance gap between processor speed and memory speed. This problem has existed since the single-core era. What makes it more challenging in this case is the shared memory hierarchy (several levels of cache memory followed by the main memory). Who shares each level of caches? Each of the computational cores discussed here targets a program (or thread or process) with different characteristics from those targeted by other computational cores. For example, a GPU requires higher bandwidth, while a traditional core requires faster access. As a result, what is needed is a memory hierarchy that reduces interference among the different cores, yet deals efficiently with the different requirements of each.

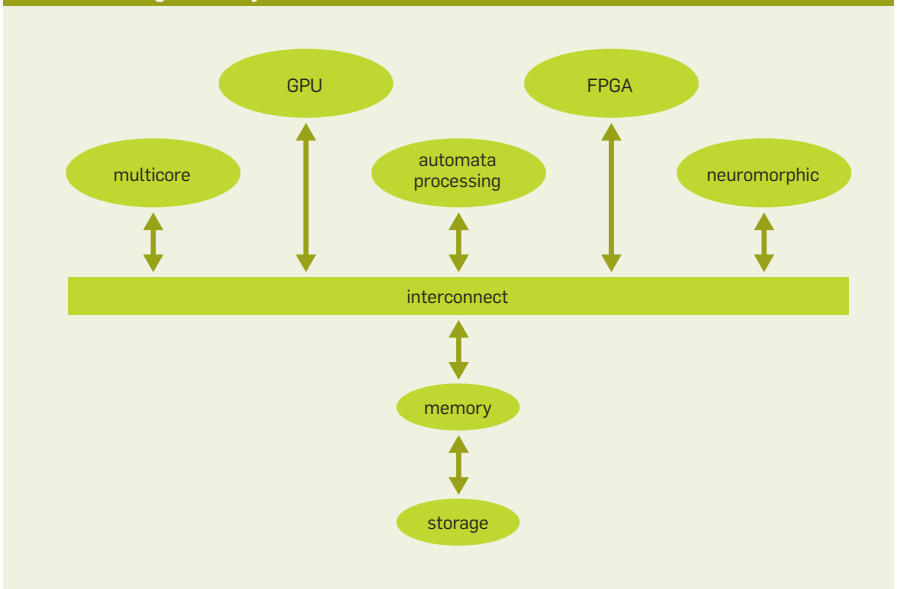
Designing such a hierarchy is far from easy, especially considering that, beside performance issues, the memory system is a nontrivial source of power consumption. This challenge is the subject of intensive research in industry and academia. Moreover, we are coming close to the era of nonvolatile memory. How can it best be used? Note here the heterogeneity in memory modules: for caches (SRAM), volatile memory (DRAM), nonvolatile memory (MRAM, STT-RAM, PCM, ReRAM, and many more technologies).

Another challenge at the hardware level is the interconnect: How should we connect the different cores and memory hierarchy modules? Thick wires dissipate less power but result in lower bandwidth because they take more on-chip space. There is a growing body of research in optical interconnect. The topology (ring, torus, mesh), material (copper, optical), and control (network-on-chip protocols) are hot topics of research at the chip level, at the board level, and across boards.

Yet another challenge is distributing the workload among the different cores to get the best performance with the lowest power consumption. The answer to this question must be found across the whole computing stack, from algorithms to process technology.

The move from a single board to multiboard and into high-performance computers also means a move from shared memory to distributed memory. This makes the interconnect and workload distribution even more challenging.

Generic heterogeneous system.



Software Challenges

At the software level, the situation is also very challenging. How are we going to program these beasts? Sequential programming is hard. Parallel programming is harder. Parallel programming of heterogeneous machines is extremely challenging if we care about performance and power efficiency. There are several considerations: how much hardware to reveal to the programmer, the measures of success,

and the need for a new programming model (or language).

Before trying to answer these questions, we need to discuss the eternal issue of productivity of the programmer vs. performance of the generated software. The common wisdom used to be that many aspects of the hardware needed to be hidden from the programmer to increase productivity. Writing in Python makes you more productive than writing in C, which is more productive than writing in assembly, right? The answer is not that easy, because many Python routines, for example, are just C wrappers. With the proliferation of heterogeneous machines, performance programmers for use by productivity programmers will create more and more libraries. Even productivity programmers, however, need to make some hard decisions: how to decompose the application into threads (or processes) suitable for the hardware at hand (this may require experimenting with different algorithms), and which parts of the program do not require high performance and can be executed in lower-power-consumption mode (for example, parts that require I/O)?

Defining the measures of success poses a number of challenges for both productivity and performance programmers. What are the measures of success of a program written for a heterogeneous machine? Many of these measures have characteristics in common with those of traditional parallel code for homogeneous machines. The first, of course, is performance. How much speedup do you get relative to the sequential version and relative to the parallel version of homogeneous computing?

The second measure is scalability. Does your program scale as more cores are added? Scalability in heterogeneous computing is more complicated than in the homogeneous case. For the latter, you just add more of the same. For heterogeneous machines, you have more options: adding more cores of some type, or more GPUs, or maybe FPGAs. How does the program behave in each case?

The third measure of success is reliability. As transistors get smaller, they become more susceptible to faults, both transient and permanent. Do you leave this issue of dealing with faults to the hardware, or system software, or shall the programmer have some say? Each strategy has its pros and cons. On the one hand, if it is left to the hardware or the system software, the programmer will be more productive. On the other hand, the programmer is better informed than the system to decide how to achieve graceful degradation in performance if the number of cores decreases as a result of failure or a thread produces the wrong result because of a transient fault. The programmer can have, for example, two versions of the same subroutine: one to be executed on a GPU and the other on several traditional cores.

Portability is another issue. If you are writing a niche program for a well-defined machine, then the first three measures are enough. But if you are writing a program for public use on many different heterogeneous computing machines, then you need to ensure portability. What happens if your code runs on a machine with an FPGA instead of a GPU, for example? This scenario is not unlikely in the near future.

The Best Strategy

Given these questions and considerations, what is the best strategy? Should we introduce new programming models (and languages), or should we fix/update current ones? Psychology has something to say. The more choices a person has, the better—until some threshold is reached. Beyond that, people become overwhelmed and will stick to whatever language they are using. But we have to be very careful about *fixing* a language. Perl used to be called a "write-only language." We don't want to fall into the same trap. Deciding which language to fix/modify is a very difficult decision, and a wrong decision would have a very high cost. For heterogeneous computing, OpenCL (Open Computing Language) seems like a good candidate for shared-memory machines, but it must be more user friendly. How about distributed memory? Is MPI (Message Passing Interface) good enough? Do any of the currently available languages/paradigms consider reliability as a measure of success?

The best scheme seems to be twofold: new paradigms invented and tested in academia while the filtering happens in industry. How does the filtering happen? It happens when an inflection point occurs in the computing world. Examples of two previous inflection points are moving from single core to multicore and the rise of GPUs. We are currently witnessing a couple of inflection points at the same time: getting close to exascale computing and the rise of the Internet of Things. Heterogeneous computing is the enabling technology for both.

Heterogeneous computing is already here, and it will stay. Making the best use of it will require revisiting the whole computing stack. At the algorithmic level, keep in mind that computation is now much cheaper than memory access and data movement. Programming models need to deal with productivity vs. performance. Compilers need to *learn* to use heterogeneous nodes. They have a long way to go, because compilers are not yet as mature in the parallel-computing arena in general as they are in sequential programming. Operating systems must learn new tricks. Computer architects need to decide which nodes to put together to get the most effective machines, how to design the memory hierarchy, and how best to connect all these modules. At the circuit level and the process technology level, we have a long wish list of reliability, power, compatibility, and cost. There are many hanging fruits at all levels of the computing stack, all ready for the picking if we can figure out the thorns. C

Related articles on queue.acm.org

Computing without Processors
Satnam Singh
http://queue.acm.org/detail.cfm?id=2000516

FPGA Programming for the Masses
David F. Bacon, Rodric Rabbah, and Sunil Shukla
http://queue.acm.org/detail.cfm?id=2443836

A Conversation with John Hennessy and David Patterson
http://queue.acm.org/detail.cfm?id=1189286

References
1. HSA Foundation; http://www.hsafoundation.com/.
2. IBM Research. The cognitive era; https://www.research.ibm.com/cognitive-computing/.
3. Micron. Automata processor; http://www.micronautomata.com/.

Mohamed Zahran is a clinical associate professor of computer science at New York University. His research interests span several areas of computer architecture and hardware/software interaction.

Copyright held by owner/author.
Publication rights licensed to ACM. $15.00

Copyright of Communications of the ACM is the property of Association for Computing Machinery and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.